Benchmarking Filtering Techniques for Entity Resolution

George Papadakis
National and Kapodistrian University of Athens, Greece
gpapadis@di.uoa.gr

Marco Fisichella
L3S Research Center, Germany
mfisichella@l3s.de

Franziska Schoger
L3S Research Center, Germany
schoger@l3s.de

George Mandilaras
National and Kapodistrian University of Athens, Greece
gmandi@di.uoa.gr

Nikolaus Augsten
University of Salzburg, Austria
nikolaus.augsten@plus.ac.at

Wolfgang Nejdl
L3S Research Center, Germany
nejdl@l3s.de

2023 IEEE 39th International Conference on Data Engineering (ICDE) | 979-8-3503-2227-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDE55515.2023.00389

***Abstract*—Entity Resolution is the task of identifying pairs of entity profiles that represent the same real-world object. To avoid checking a quadratic number of entity pairs, various filtering techniques have been proposed that fall into two main categories: (i) *blocking workflows* group together entity profiles with identical or similar signatures, and (ii) *nearest-neighbor methods* convert all entity profiles into vectors and identify the closest ones to every query entity. Unfortunately, the main techniques from these two categories have rarely been compared in the literature and, thus, their relative performance is unknown. We perform the first systematic experimental study that investigates the relative performance of the main representatives per category over numerous established datasets. Comparing techniques from different categories turns out to be a non-trivial task due to the various configuration parameters that are hard to fine-tune, but have a significant impact on performance. We consider a plethora of parameter configurations, optimizing each technique with respect to recall and precision targets. Both schema-agnostic and schema-based settings are evaluated. The experimental results provide novel insights into the effectiveness, the time efficiency and the scalability of the considered techniques.**

***Index Terms*—nearest neighbors, record linkage, deduplication**

I. Introduction

Entity Resolution (ER) is a well-studied problem that aims to identify so-called *duplicates* or *matches*, i.e., different entity profiles that describe the same real-world object [1]. ER constitutes a crucial task in a number of data integration tasks, which range from Link Discovery for interlinking the sources of the Linked Open Data Cloud to data analytics, query answering and object-oriented searching [2]–[4].

Due to its quadratic time complexity, ER typically scales to large data through a Filtering-Verification framework [1], [3]. The first step (*filtering*) constitutes a coarse-grained, rapid phase that restricts the computational cost to the most promising matches, a.k.a. *candidate pairs*. This is followed by the *verification* step, which examines every candidate pair to decide whether it is a duplicate.

Verification is called *matching* in the ER literature [2], [3], [5]. Numerous matching methods have been proposed; most of them rely on similarity functions that compare the textual values of entity profiles [2], [3], [5], [6]. Early attempts were rule-based, comparing similarity values with thresholds, but more recent techniques rely on learning, i.e., they usually model matching as a binary classification task (`match`, `non-match`) [6]. Using a labelled training dataset, supervised, active, and deep learning techniques are adapted to ER [7]. In some cases, a clustering step is subsequently applied on the resulting similarity scores to refine the output [8].

In this work, we are interested in filtering methods, which significantly reduce the search space of ER. The performance of a filtering technique is assessed along three dimensions: recall, precision and time efficiency. A good filter stands out by:

1) *High recall.* The candidate pairs should involve many duplicates to reduce the number of false negatives.
2) *High precision.* The candidate pairs should involve few false positives (i.e., non-matching pairs) to significantly reduce the search space.
3) *Low run-time.* The overhead added by the filtering step to the overall run-time of ER should be low.

Ideally, filtering should also be directly applicable to the input data. For this reason, we exclusively consider methods that require no labelled training instances, which are often not available or expensive to produce [9]. We also focus on techniques for textual entity profiles. These methods are organized into two types [10]:

1) *Blocking workflows* extract one or more signatures from every entity profile and form clusters of profiles with identical or similar signatures [3], [11], [12].
2) *Nearest neighbor (NN) methods* index part or all of the input data and treat every entity profile as a query, to which they return the most similar candidates. They rely on sparse [13], [14] or dense numeric vectors [15].

Although the two types of filtering techniques follow very different approaches, they all receive the same input (the entity profiles) and produce the same output (candidate pairs).

To the best of our knowledge, no previous work systematically examines the relative performance across the two different kinds of methods. The main blocking methods are empirically evaluated in [11], [12], [16]. The studies on sparse [13], [14], [17] and dense vector-based NN methods [15] evaluate run-time and approximation quality, but do not evaluate the performance on ER tasks (cf. Section II). Even in the few cases where blocking is used as baseline for an NN method (e.g., Standard/Token Blocking is compared to DeepBlocker in [9]), the comparison is not fair: the blocking method is treated

2375-026X/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDE55515.2023.00389

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.

as an independent approach, instead of applying it as part of a blocking workflow, which is common in the literature [11].

Comparing techniques from different categories turns out to be a challenging task, due to their fundamentally different functionality and the diversity of configuration parameters that significantly affect their performance. Yet, there is *no* systematic fine-tuning approach that is generic enough to apply to all filtering methods – e.g., the step-by-step configuration optimization in [11] applies to blocking pipelines, but not to the single-stage NN methods.

In this work, we perform the first thorough and systematic experimental study that covers both types of filtering methods. To ensure a fair comparison, every approach is represented by its best performance per dataset, as it is determined after an exhaustive fine-tuning with respect to a common performance target that considers several thousand different parameter configurations. This approach is applied to 5 blocking workflows and 8 NN methods over 10 real-world datasets. For each filter type, we also consider two baseline methods with default parameters. We explore both *schema-agnostic* and *schema-based settings*: the latter focus exclusively on the values of the most informative attribute, while the former take into account all information within an entity, essentially treating it as a long textual value – as a result, they are inherently applicable to datasets with heterogeneous schemata.

Overall, we make the following contributions:

- We perform the first thorough experimental analysis on filtering techniques from different categories. We evaluate 14 state-of-the-art filters and 4 baselines on 10 real-world datasets in both schema-based and schema-agnostic settings.
- Our configuration optimization process enables the meaningful comparison of blocking workflows and NN methods in the context of serial processing (single-core execution).
- We present a qualitative analysis of the filtering techniques based on their *scope* and *internal functionality*.
- We perform a thorough scalability analysis that involves seven synthetic datasets of increasing size.
- Our work provides new insights into the relative performance and scalability of the considered techniques. We show that the blocking workflows and the cardinality-based sparse NN methods consistently excel in performance.
- Two of the tested NN methods, SCANN and kNN-Join, are applied to ER for the first time. SCANN is one of the most scalable techniques, and kNN-Join one of the best performing ones, while sticking out by its intuitive fine-tuning.
- All code and data used in this work are publicly available through a new, open initiative that is called Continuous Benchmark of Filtering methods for ER: https://github.com/gpapadis/ContinuousFilteringBenchmark.

The main part of the paper is structured as follows: Section II discusses the related works, while Section III defines formally the configuration optimization task. We elaborate on the filtering methods in Section IV and present their qualitative and quantitative analyses in Sections V and VI, resp. Section VII concludes with the main findings of our experimental analysis along with directions for future research.

II. Related Work

There has been a plethora of works examining the relative performance of blocking methods. The earliest systematic studies were presented in [3], [12]. They focus exclusively on schema-based settings in combination with several user-defined parameter configurations. They also consider exclusively the first step of blocking workflows: block building.

These studies were extended in [16], which examines the same block building methods and configurations, but applies them to schema-agnostic settings, too. The experimental outcomes suggest that recall raises significantly, when compared to the schema-based settings, while requiring no background knowledge about the given data and the quality of its schema. They also suggest that the sensitivity to parameter configuration is significantly reduced.

Building on these works, the experimental analysis in [11] examines the relative performance of the blocking workflows in schema-agnostic settings. In particular, it considers blocking workflows formed by exactly three steps: block building, block filtering and comparison cleaning. In our work, we extend this analysis by considering the top performing block building methods, all of which cluster together entities that share identical signatures. We combine them with three consecutive, but optional steps: block purging, block filtering and comparison cleaning. These steps give rise to seven different filtering pipelines, out of which only one was examined in [11].

Our work also differs from [11] in that it has only one dataset in common – excluding the scalability ones. Most importantly, [11] optimizes the configuration parameters in a heuristic step-by-step manner: first, the performance of block building is heuristically optimized and then, block filtering is heuristically fine-tuned by receiving as input the output of optimized block building as so on for comparison cleaning. In contrast, we consider a holistic approach to configuration optimization, simultaneously fine-tuning all steps in a blocking workflow. As explained in [18], [19], this approach consistently outperforms the step-by-step fine-tuning, because it is not confined to local maxima per workflow step, while it considers a significantly larger set of possible configurations.

Finally, we also go beyond [3], [11], [11], [12] in two more ways: (i) we systematically fine-tune blocking workflows in the context schema-based settings, and (ii) we compare blocking workflows with sparse and dense NN methods.

The sparse vector-based NN methods essentially correspond to similarity joins. The relative performance of the main methods is examined in [13], [14], [17] with respect to runtime. Recall and precision are not considered, because they are identical across all approaches, i.e., all methods retrieve the same pairs of entities that exceed a similarity threshold. These pairs, which are not necessarily matching, have not been evaluated with respect to the recall and precision of ER. As a result, these studies are not useful in assessing the performance of string similarity joins for ER. Note also that none of the sparse NN methods we consider has been examined in prior experimental analyses. The reason is that the local kNN-Join

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.

lies out of the focus of [13], [14], [17], while the range joins for ER involve very low similarity thresholds, unlike the string similarity joins examined in [13], [14], [17].

The dense vector-based NN methods are experimentally compared in [15], but the evaluation measures are restricted to throughput, i.e., executed queries per second, and to recall, i.e., the portion of retrieved vectors that are indeed the nearest ones with respect to a specific distance function (e.g., the Euclidean one). This is different from ER recall, as the closest vectors are not necessarily matching. Nevertheless, we rely on the experimental results of [15] in order to select the top-performing dense NN methods: Cross-polytope and Hyperplane LSH, FAISS and SCANN. We also consider MinHash LSH, a popular filtering technique [10], and DeepBlocker, the most recent learning-based approach, which consistently outperforms all others [9].

III. Preliminaries

We define an entity profile e_i as the set of textual name-value pairs, i.e., $e_i = \{\langle n_j, v_j \rangle\}$, that describes a real-world object [11], [16]. This model covers most established data formats, such as the structured records in relational databases and the semi-structured instance descriptions in RDF data. Two entities, e_i and e_j, that pertain to the same real-world object ($e_i \equiv e_j$) are called *duplicates* or *matches*.

ER is distinguished into two main tasks [3], [5]:

1) *Clean-Clean ER* or *Record Linkage*, which receives as input two sets of entity profiles, $\mathcal{E}_1$ and $\mathcal{E}_2$ that are individually duplicate-free, but overlapping, and
2) *Dirty ER* or *Deduplication*, whose input comprises a single set of entity profiles, $\mathcal{E}$, with duplicates in itself.

In both cases, the output consists of the detected duplicate profiles. In the context of Clean-Clean ER, the filtering methods receive as input $\mathcal{E}_1$ and $\mathcal{E}_2$ and produce a set of candidate pairs $\mathcal{C}$, which are highly likely to be duplicates and should be analytically examined during the verification step. To measure the **effectiveness** of filtering, the following measures are typically used [5], [11], [12], [16], [20]:

1) *Pair Completeness (PC)* expresses *recall*, estimating the portion of the duplicate pairs in $\mathcal{C}$ with respect to those in the groundtruth: $PC(\mathcal{C}, \mathcal{E}_1, \mathcal{E}_2) = |\mathcal{D}(\mathcal{C})|/|\mathcal{D}(\mathcal{E}_1 \times \mathcal{E}_2)|$, where $\mathcal{D}(x)$ denotes the set of duplicates in set x.
2) *Pairs Quality (PQ)* captures *precision*, estimating the portion of comparisons in $\mathcal{C}$ that correspond to real duplicates: $PQ(\mathcal{C}) = |\mathcal{D}(\mathcal{C})|/|\mathcal{C}|$.

All measures result in values in the range $[0, 1]$, with higher values indicating higher effectiveness. Note that there is a trade-off between PC and PQ: the larger C is, the higher PC gets at the cost of lower PQ, and vice versa for a smaller set of candidates. The goal of filtering is to achieve a good balance between these measures.

In this context, we formalize the following configuration optimization task, which enables the comparison of fundamentally different filtering techniques on an equal basis:

Problem 1 (Configuration Optimization): Given two sets of entity profiles, $\mathcal{E}_1$ and $\mathcal{E}_2$, a filter method, and a threshold τ on recall (PC), configuration optimization fine-tunes the parameters of the filtering method such that the resulting set of candidates $\mathcal{C}$ on $\mathcal{E}_1$ and $\mathcal{E}_2$ maximizes PQ for $PC \geq \tau$.

Note that we set a threshold on recall, because ER solutions typically consist of two consecutive steps: Filtering and Matching. The recall of the former step determines the overall recall of ER, since the duplicate pairs that are not included in the resulting set of candidates C cannot be detected by most matching methods. This applies both to Clean-Clean ER, where there is no transitivity, due to the 1-1 matching constraint, and to Dirty ER, where the matching algorithms typically consider local information, e.g., the (deep) learning-based methods that treat matching as a binary classification task [21], [22]. As a result, we chose $\tau = \mathbf{0.9}$ as the PC threshold that ensures high recall for the overall ER process. Preliminary experiments demonstrated that a lower threshold, e.g., 0.85, didn't alter the relative performance of the considered techniques. Besides, in more than 40% of considered cases, the filtering techniques address Problem 1 with $PC \geq 91\%$, i.e., much higher than our threshold.

Regarding **time efficiency**, the *run-time* (RT) measures the time between receiving the set(s) of entity profiles as input and producing the set of candidate pairs as output. RT should be minimized to restrict the overhead of filtering on ER.

IV. Filtering Methods

A. The two paradigms of filtering

Blocking methods first associate every input entity with one or (usually) more signatures and then, they cluster together entities with identical or similar signatures into blocks. Every pair of entities that appears in at least one block is considered a matching candidate. The resulting blocks contain two unnecessary types of candidates, i.e., pairs whose verification lowers precision, without any benefit for recall:

1) The *redundant* candidates are repeated across different blocks, because every entity typically participates in multiple ones and, thus, the blocks are overlapping.
2) The *superfluous* candidates involve non-matching entities.

To eliminate the former and reduce the latter, block and comparison cleaning are applied to the initial blocks, restructuring them based on global patterns [11], [23]. Figure 1 depicts the complete workflow for blocking [10], [24]. Initially, a set of blocks is created by at least one block building method. The initial block collection is then processed by two coarse-grained block cleaning techniques, Block Purging and Block Filtering. Both produce a new, smaller block collection, B' and B'', respectively, but are optional and can be omitted, e.g., in the case of schema-based blocks with low levels of redundant and/or superfluous comparisons. Finally, a comparison cleaning technique is applied, whose fine-grained functionality decides for individual comparisons whether they should be retained or discarded; in this mandatory step, at least the redundant candidates are discarded, but the superfluous ones are also subject to removal.

A fundamentally different approach is followed by NN methods. Instead of extracting signatures from the input entity

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.

Description-Similarity Rules: Towards Flexible Feature Engineering for Entity Matching

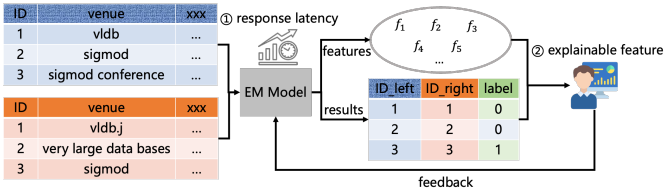
Yafeng Tang[1], Zheng Liang[1], Hongzhi Wang[1,✉], Xiaoou Ding[1], Tianyu Mu[1], Huan Hu[2]
[1] School of Computer Science, Harbin Institute of Technology, Harbin, China
[2] Huawei Cloud Computing Technologies Co., Ltd, Hangzhou, China
tangyf@stu.hit.edu.cn,{lz20, wangzh, dingxiaoou,mutianyu}@hit.edu.cn, huhuan18@huawei.com

***Abstract*—Entity Matching (EM) is a crucial task in data integration. Compared to deep learning-based EM solutions, tree-based machine learning models are more computationally effective and explainable, making them more applicable in real-world EM scenarios. However, Random Forest-based EM methods select features with a static feature engineering rule set for all attributes. Consequently, they suffer model retraining cost to select features, and can hardly customize to different EM tasks. To tackle this problem, we propose Description-Similarity Rules (DSR) for EM feature engineering. DSR introduces diverse attribute value distribution metrics and data-driven thresholds to traditional EM feature engineering rules. Unfortunately, both the DSR search space and its online model retraining costs are exponential. By pushing the model retraining to the offline stage, our DSR set mining algorithm is one order of magnitude faster than the baseline algorithms, taking only seconds for online selection. Empirically, DSR outperforms the feature engineering methods by 3.75% on average F1 score, while reaching the state-of-the-art EM performance on several datasets.**

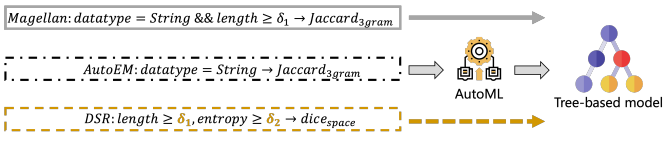
***Index Terms*—Entity Matching, Feature Engineering, Rule Mining**

I. Introduction

Entity matching [1]–[3] is the problem of identifying co-referent manifestations that refer to the same real-world entity from different data sources. For example, attributes of the same person from multiple sources are often complicated [4] in the real world. EM can improve data quality by reducing such duplicates among tables [5].



(a) An interactive EM pipeline



(b) Rule-based EM model

Fig. 1: A typical EM pipeline.

✉ Hongzhi Wang is the corresponding author.

EM has been an open problem for decades. Conventional EM solutions are based on similarity functions, which quantify the similarity between pairwise attribute values of an entity [6]. Rule-based [7]–[10] EM methods use certain thresholds for attribute similarities to make matching decisions, where experts specify the thresholds (e.g., the value of δ_1 in Figure 1(b)). Machine learning-based methods [6] train machine learning models on pairs of labeled entities with similarities as features. They search for the feature combination with the best evaluation metric (e.g., F1 score). Deep Learning methods [1], [3], [11]–[14] incorporate domain knowledge in language-model-based encoders. However, real-world EM tasks pose flexibility requirements beyond the grasp of these methods, which can be specified in the following aspects [15]. (1) **Response latency.** Entity Matching is rarely a one-time job. As Figure 1(a) shows, a typical EM pipeline requires data scientists to go through the cycle of exploring data [10], changing EM the solution, and checking results several times [16]. Also, EM becomes even more time-intensive on multi-source data which need to be split and matched separately. Methods with low response latency are necessary for refining and validating EM solutions in a pipeline as Figure 1. (2) **Explainable features.** To understand and reconfigure a machine learning model in the EM pipeline, one needs to understand how the model is influenced by its input, i.e., the feature value. Even for explainable EM models like Random Forest, system-selected features still need to be traceable from data [17].

To the best of our knowledge, few EM solutions can address the above two issues simultaneously. Rule-based methods like Autofuzzyjoin [18] may not perform well with less structured datasets and long textual data since they rely on limited word level similarity metrics and TF-IDF scores [19]. State-of-the-art deep learning methods like Sudowoodo [13] make it difficult for users to understand why entities are matched or unmatched. They often require a high cost for debugging the model: try numerous configurations at each iteration, and need expert experience to make decisions. Mainstream machine learning-based methods are easier to understand, but their efficiency is limited by searching models and features. Therefore, to meet the flexibility demands mentioned above, lightweight machine learning models such as Random Forest (RF) [20] have been generally adopted for EM [21] in commercial data integration pipelines [6], [15], [22] for their good explainability and satisfactory performance.

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00008

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00008

Example 1. *Consider a training process of a Random Forest-based EM model, a data scientist needs to construct features (i.e. similarities) for each attribute pair.*

In Figure 1(b), Magellan [23] simply uses length and data types to recommend similarities, so the selected features result in poor performance (Section V). AutoML-EM [6] relaxes the rule set of Magellan and recommends redundant features, which are subsequently trimmed by auto-sklearn [24]. The general feature selection methods [8], [25]–[29] search for the optimal feature at the cost of exhaustive model retraining. Considering efficiency, in this paper, we focus on rule-based feature selection methods.

Challenges. From Example 1, we can see that the bottleneck of the RF-based EM pipeline is the drawback of rule-based feature selection, which can be summarized as follows.

Exhaustive Rule Candidate Space. In a typical EM pipeline, rule-based similarity selection methods typically cost extensive computational resources as expert knowledge is unavailable. The threshold of descriptions [1] can take any value within a continuous range. Actually, we need to consider rules with conjunction of multiple descriptions as antecedents, and all similarities can be consequent. As a result, the rule candidate space is overwhelming, and searching for each rule candidate requires an EM model retraining. What's worse, an EM pipeline mostly lacks labeled data, so the hyperparameter search algorithms [30] fail to solve this problem.

Rule set is too rigid. The rule set of existing solutions only considers `length` and `datatype` of attribute pair, while the threshold is fixed for all EM tasks. They fail to offer users enough flexibility to give feedback and perform an EM task at hand. Consequently, such rigid rule sets with fixed thresholds recommend too few or insufficient features for complex datasets (as shown in Example 1). Users can barely understand EM data and give constructive feedback to the pipeline in such cases.

Description-Similarity Rule. Rule-based FE methods fall short in practice due to the above limitations. To unlock the flexibility of EM models, we need to design a flexible substitute of feature engineering rules without too much computational overhead. Motivated by this, we propose the Description-Similarity Rule (DSR). DSR helps users in EM pipeline to better understand data with the newly introduced "description". Also, DSR has a flexible threshold, which can be discovered from data without expert knowledge.

Contribution. The contributions of our work can be summarized as follows.

• We propose the Description-Similarity Rule (DSR), a set of data-driven rules for flexible Feature Engineering on Entity Matching models. We design *descriptions* to support common data types including sets, vectors, strings, and boolean data.

• To discover the optimal DSR set for a given EM task, we design a DSR mining algorithm accelerating the baseline approach by one order of magnitude (Section IV). Also, we address the problem of exponential online retraining by sampling historical datasets with a theoretical guarantee on error probability(Section III).

• Empirically, DSR beats the state-of-the-art EM solution by 3.75% on average F1 score. DSR is also statistically comparable to black-box Deep Learning methods (Section V).

[1]Description is a set metric to compute certain attribute value distributions. We will go into details in Definition 2

TABLE I: Frequently used notations.

Symbol	Meaning
T_1, T_2	A pair of tables for entity matching
$A_1, A_2, \ldots$	The attribute set of tables
t_1, t_2	An entity pair from T_1 and T_2
M	The entity matching model
p	The evaluation metric for the EM task
$D = \{d_1, d_2, d_3, d_4\}$	A set of descriptions
D'	A subset of descriptions
$S = \{S_1, S_2, \ldots\}$	A set of similarities
$R = \{r_1, r_2, \ldots\}$	A set of description-similarity rules
δ_i	The threshold for i-th description
θ	The threshold for lowest acceptable performance
ϕ	An operator
$dom(d_i)$	The domain of valid value of description
E	The expectation of a variable
n	The budget of EM model retraining times

II. Overview

In this section, we define essential concepts for Description-Similarity Rules (Section II-A), elaborate the design details of *Descriptions* (Section II-B), present the DSR recommendation problem(Section II-C), and give an overview of our solution (Section II-D).

A. Background

Entity Matching solutions for structured data consist of at least two phases: blocking and matching [31], and often assume that upstream tasks have prepared datasets with aligned attributes [6], [32]. In this paper, we focus on the feature engineering of entity matching for relational data and treat entity blocking as an orthogonal problem.

Definition 1. ***Entity Matching (EM):*** *Given two relational tables T_1 and T_2 with the aligned attribute set $A_1, A_2, ...$ with the same schema, Entity Matching determines whether each pair of records $t_1, t_2 \in T_1 \times T_2$ refers to the same entity in the real world.*

In this paper, we focus on the Random Forest(RF)-based EM solver, as RF is a widely-adopted machine learning model in real-world EM applications [6], [12], [22], [33]. For example, consider the entity pair with $ID = 3$ in Figure 1(a), "sigmod conference" and "sigmod" have a Jaccard similarity of 0.5, which is used by RF-based EM solvers as one of the features to represent this entity pair for RF training. Therefore, the core of *RF-based EM* is **similarity selection** [7], [34], [35], and we follow SOTA RF-based EM methods [6], [23] to exploit similarity selection rules.

A similarity function quantifies the similarity between two strings (or numeric or Boolean values). There are four types of similarity functions: set-based, sequence-based, vector-based,

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.

and number-based. Next, we will introduce how we design *descriptions* to support the recommendation of these functions.

B. Preliminaries

Our intuition is that general FE methods [36], [37] or intuitive EM methods [6], [23] ignore latent attribute value statistics during similarity recommendation, thus cannot obtain ideal performance on EM task (see Section V for details). Therefore, we propose a new concept called *description* to overcome their drawbacks. For each *description*, we show their relationship with similarity functions.

Definition 2. ***Description (D):*** *Let $T_1.A_i$ and $T_2.A_i$ denote the set of the attribute value A_i of all entities in T_1 and T_2, respectively, the description $d \in D, D = \{d_1, d_2, d_3, d_4\}$ on attribute $d(A_i)$ can be computed as follows:*[2]

- `length`*: For sequence-based similarities such as Jaccard and Jaro [36], we present* `length` *as the length of the string in attribute A.*
- `AvgWordNum(AWN)`*: For set-based similarities, we present* `AWN` *to describe strings including separators.* `AWN` *is associated with similarity functions such as the dice-coefficient [38] and Jaccard Index [39].*

$d_2(AvgWordNum)$: Suppose N is a tokenizer. $N(A)$ denotes the tokenized wordBag after employing N on attribute A,

$$d_2(A_i) = |N(A_i)|$$

A tokenizer can be a publicly available tool [40], or any common delimiter such as spaces, commas, semicolons, etc.

- `Entropy`*: For vector-based similarities [41], we design* `entropy` *to convert strings into document vectors [42], especially for long texts.*

$d_3(entropy)$: For each attribute A, we treat A as a document, and $v(A_i)$ represents the vocabulary of A_i. The `entropy` *of A is calculated as follows:*

$$d_3(A_i) = -\sum_{j=1}^{|v(A_i)|} p_{ij} \cdot \log_2(p_{ij}) \tag{1}$$

where p_{ij} is the probability of the j-th word occurring in document A_i.

- `Index of Coincidence(IoC)`*: For number-based similarities, we present* `IoC` *[43] to identify the distribution of characters,* ***especially where numerical and textual values are mixed****.*

$d_4(IoC)$: We denote $v(A_i)$ as the vocabulary of A_i, and f_c as the frequency of character c in A_i. The `IoC` *of A_i is as follows:*

$$d_4(A_i) = \sum_{c \in v(A_i)} \frac{f_c(f_c - 1)}{|A_i|(|A_i| - 1)}$$

We explain the design of descriptions with Example 2. In our technical report [45], we also provide a formal basis to show that descriptions are actually building blocks of the equation of all four types of similarities.

[2]notations in Definition 2 will not be presented in Table I.

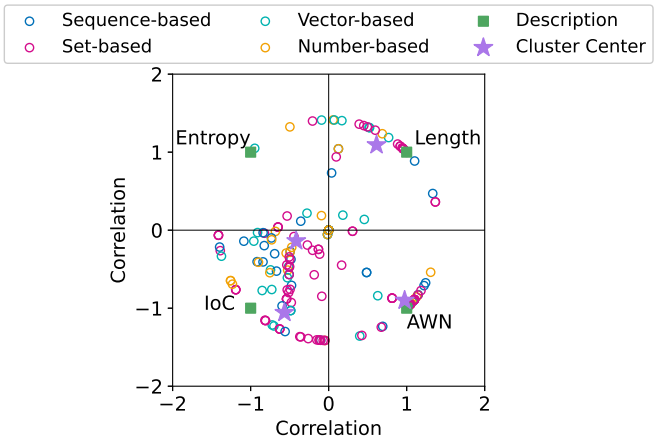


Fig. 2: The correlation diagram between four types of similarity functions (circles) and our descriptions (squares) on the Fodors-Zagats dataset [23] (better viewed in color). The purple stars represent the cluster centers of similarity functions, derived from the K-Means algorithm [44].

Example 2. *In Figure 2, we visualize the correlation between similarities and descriptions on the Fodors-Zagats dataset [23].First, we fix our four descriptions (square) in the center of each quadrant. Then, for each similarity, we calculate its Pearson correlation coefficient with each of the four descriptions and find a point (circle) whose Euclidean distance to each description is proportional to the corresponding coefficient. Note that every quadrant contains quite a few scattered circles. In addition, we apply K-Means clustering for the circles (similarities) and mark the center of the cluster with stars. Another interesting discovery is that three out of four stars land somewhere near a square. That means each description "attracts" considerable amount of similarities.*

As shown in Figure 2, descriptions can serve as the effective precomputation for all similarities. This empirical observation motivates us to explore the idea of recommending similarities based on four descriptions for a given EM task.

Under these definitions, we formally introduce DSR.

Definition 3. ***Description-Similarity Rule:*** *Given a Description set D and a Similarity Function set $S = \{S_1, S_2, \ldots\}$, the Description-Similarity-Rule (DSR) r is:*

$$r: \bigwedge_{d_i \in D' \subseteq D} d_i \geq \delta_i \rightarrow S_j \tag{2}$$

For example, a valid DSR can be written as:

$$(length \geq 10) \wedge (entropy \geq 2.92) \rightarrow Levenshtein$$

Given an entity pair (t_1, t_2), for each attribute A_i, DSR first checks whether its `length` ≥ 10, and then checks whether its `entropy` ≥ 2.92. If all conditions are met, DSR computes the Levenshtein Similarity between $t_1[A_i]$ and $t_2[A_i]$ as the feature. Rules in the DSR set will be traversed, and all similarity functions that meet the *description* constraints will be added to the feature set for the current attribute.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.



Enriching Relations with Additional Attributes for ER

Mengyi Yan
Beihang University, China
yanmy@act.buaa.edu.cn

Wenfei Fan
Shenzhen Institute of Computing Sciences, China
University of Edinburgh
United Kingdom
Beihang University, China
wenfei@inf.ed.ac.uk

Yaoshu Wang
Shenzhen Institute of Computing Sciences, China
yaoshuw@sics.ac.cn

Min Xie*
Shenzhen Institute of Computing Sciences, China
xiemin@sics.ac.cn

ABSTRACT

This paper studies a new problem of relation enrichment. Given a relation D of schema R and a knowledge graph G with overlapping information, it is to identify a small number of relevant features from G, and extend schema R with the additional attributes, to maximally improve the accuracy of resolving entities represented by the tuples of D. We formulate the enrichment problem and show its intractability. Nonetheless, we propose a method to extract features from G that are diverse from the existing attributes of R, minimize null values, and moreover, reduce false positives and false negatives of entity resolution (ER) models. The method links tuples and vertices that refer to the same entity, learns a robust policy to extract attributes via reinforcement learning, and jointly trains the policy and ER models. Moreover, we develop algorithms for (incrementally) enriching D. Using real-life data, we experimentally verify that relation enrichment improves the accuracy of ER above 15.4% (percentage points) by adding 5 attributes, up to 33%.

PVLDB Reference Format:
Mengyi Yan, Wenfei Fan, Yaoshu Wang, and Min Xie. Enriching Relations with Additional Attributes for ER. PVLDB, 17(11): 3109 - 3123, 2024.
doi:10.14778/3681954.3681987

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/SICS-Fundamental-Research-Center/Enrichment.

1 INTRODUCTION

When we talk about incomplete information, we typically refer to (null) values and tuples missing from a relation D of schema R. However, for an application at hand, schema R may be incomplete; it "may not have all attributes required for analysis" [131]. As a consequence, the tuples in D do not have enough information for the application, *e.g.,* causal inference [131], aggregate SQL queries [84, 130] and movie popularity classification [33]. As exemplified in [131], a data analyst in the WHO organization aimed to estimate the effect of a mask policy on the coronavirus mortality rate, but found critical attributes (*e.g.,* weather that affects people's willingness to wear masks) not included in the data [131].

*Corresponding author
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 11 ISSN 2150-8097.
doi:10.14778/3681954.3681987

Table 1: An Example of Person **Table**

tid	name	gender	email	address	city	age	spouse_name	eid
t_1	James Davis	M	james@example.com	18 Elmwood Rd	Chicago	45	Ava Davis	e_1
t_2	John Wilson	M	john@example.com	18 Maple Avenue	Houston	null	Ava Wilson	e_2
t_3	Ava Davis	F	ava@example.com	12 Pine Lane	Boston	36	John Wilson	e_3
t_4	Ava Wilson	F	a.Wat@example.com	18 Maple Avenue	Houston	36	John Wilson	e_3
t_5	Ava Davis	F	ad123@example.com	18 Elmwood Rd	Chicago	42	James Davis	e_4

Attributes are missing from schema R for several reasons [91, 114]. (1) People may lack full knowledge of desired functionality in large-scale applications. (2) The application world constantly evolves, necessitating enhancements. (3) The scale of tasks often requires incremental design and commissioning. Regardless of the reasons, relation D often misses features needed for our application.

In this paper, we focus on entity resolution (ER) as our target application, which has been commonly used in, *e.g.,* e-commerce and financial institutions. As an example, the insurance industry employs ER as a routine operation in detecting identity fraud, which was responsible for over $17 billion stolen from U.S. consumers in 2017 [1]. As reported in [3], fraudsters forge fake identities. Such a fraud is hard to detect, since it can use information stolen from a real person, and a mix of real and synthetic data [6]. There are usually no unique identities as references, since collecting them is "time-consuming, adds friction to the customer journey, and is also an easy check to bypass" [12]. Similar problems also exist in card fraud (*e.g.,* duplicate applications for gift/credit cards) or money laundering (*e.g.,* move money from one account into another) [5, 111].

To detect fraud, the insurance industry has been advocating "to incorporate external information into the identity verification process" [6]. This has been practiced by, *e.g.,* SEON [13] and SIFT [14].

Example 1: Consider Table 1 with 5 tuples t_1-t_5 for 4 persons e_1-e_4, who are applying for promotional gift cards, where each person is limited to one card (*e.g.,* eGift of Starbucks [16]). The tuples have a schema with attributes name, gender, email, address and city; the store allows one person to have, *e.g.,* multiple different emails.

A not-so-sophisticated ER method $\mathcal{A}_{\mathsf{ER}}$ may predict that t_3 and t_5 are the same person (a false positive, FN), since they have exactly the same name. It may also miss the true match of t_3 and t_4 (a false negative, FN), due to different names, emails and addresses, although Ava Wilson was named Ava Davis before her marriage, and she issued a duplicate application using her old identity.

Fortunately, additional attributes can help us reduce FPs/FNs of ER. (a) As remarked in [131], age or ethnicity are often missing in person datasets. If additional attribute age is available, it provides a strong evidence for $\mathcal{A}_{\mathsf{ER}}$ to tell that t_3 and t_5 are mismatched, since they have different ages. (b) If we further enrich the schema with attribute spouse_name, (*e.g.,* by social networks [12]), $\mathcal{A}_{\mathsf{ER}}$ can identify t_3 and t_4, since they are married to the same person. □

Missing attributes are as damaging as missing data, but it has not received much attention. While there has been a host of work on missing data imputation [25, 26, 29, 40, 49, 50, 54, 55, 66, 77, 100, 105, 116, 128, 129], no prior work has systematically studied how to enrich incomplete schema in order to improve ER accuracy. They either do not target downstream applications or is not developed for ER and thus, miss distinguishing attributes for ER (see Section7).

To enrich dataset D of schema R for ER, we can extract information from external sources *e.g.,* text [63, 64], information space [43], XML [126], data warehouse [22] and Web [59]. In fact, knowledge enrichment has been practiced in medicine [112], network [133], e-commerce [15], recommendation [113] and text generation [132]. Among them, knowledge graphs (KGs) are particularly promising for ER. KGs link related entities and disambiguate entities with similar names [17]; moreover, their attributes are already reconciled and typically highly informative [130, 131], making it feasible to improve the ER accuracy. In particular, financial institutions have been using KGs to detect various types of fraud [5]; indeed, "using a KG for ER, the company can easily detect clusters of fake claims" [3].

In light of this, we study relation enrichment for ER by referencing a KG G. We consider *reliable* G, *i.e.,* a KG that is relatively clean and complete. Several popular KGs are in place, *e.g.,* general-purpose Wikidata [9] and domain-specific DRKG [71]. The issue is highly nontrivial. It requires us to link tuples in D with vertices in G for enrichment. Worse still, a KG typically maintains all sorts of properties of entities and their links to provide a comprehensive picture. If we enrich schema R with all such properties, it may hamper the accuracy of ER. As evidenced by [34], only *relevant* attributes contribute positively to identifying true positives, and "attributes containing null values may affect negatively the ER result". This motivates us to enrich schema with bounded relevant features.

Moreover, real-life datasets and KGs constantly change, *e.g.,* Wikidata publishes hundreds of live updates every minute [8] and the IMDB we used is refreshed daily [11]. In particular, financial institutions often require real-time detection of, *e.g.,* fraud in the online payment of credit cards [18]. It is too costly to conduct relation enrichment starting from scratch in response to the updates. These motivate the need for incremental enrichment.

To make the idea work, several questions have to be answered. What distinguishing features should we add to R to best improve the ER accuracy? For a tuple t in D, where can we find the additional attributes from KG G to complement t? How can we incrementally maintain the enriched D in response to updates to D and G?

Contributions & Organization. This paper studies relation enrichment for improving the accuracy of ER models. Consider a relation D of schema R and assume a reliable knowledge graph G.

(1) An enrichment scheme (Section 3). We formulate *the problem of relation enrichment for ER*. Given a black-box ER model $\mathcal{A}_{\mathsf{ER}}$ and a parameter m, it is to extract at most m features from knowledge graph G and extend schema R with the features as additional attributes, in order to maximize the accuracy of $\mathcal{A}_{\mathsf{ER}}$. We separate schema enrichment from data enrichment, and show that the former is NP-complete and the latter is in PTIME. This said, we propose a scheme ENRICH for enriching both schema R and relation D.

(2) Schema enrichment (Section 4). We propose a method for enriching schema R for ER under ENRICH. We develop a method for heterogeneous entity resolution (HER) to identify top-ranked matches (tuples and vertices) across relation D and KG G. We learn a policy via reinforcement learning to extract at most m features that extends R to schema R_G. Each feature is fetched by a path in G. We pick features that are as diverse from the existing attributes of R as possible, yield as few null values as possible, and maximumly improve the accuracy of ER. To make the policy robust to different data distributions, we jointly train the policy and the model $\mathcal{A}_{\mathsf{ER}}$.

(3) Data enrichment (Section 5). Under ENRICH, we develop algorithms for enriching the relation D to an instance D_G of schema R_G. We support both a batch mode and an incremental mode. In the batch mode, we extend each tuple t of D by identifying vertices v in G that refer to the same entity as t via HER, traversing paths from v to extract the additional features, and adding the features to t. In the incremental mode, we dynamically maintain D_G in response to updates to both relation D and graph G. To scale with large G and D, we parallelize the algorithms and show their parallel scalability, *i.e.,* they guarantee to reduce runtime when more resources are used [81]. We defer the parallelization to [10] for the lack of space.

(4) Experimental study (Section 6). Using real-life data and benchmarks, we empirically find the following. (a) Relation enrichment improves the accuracy of ER models by 15.4% on average, up to 33%, by adding 5 attributes. (b) It is on average 5.2% (resp. 14.6%) more accurate than ML models for feature augmentation (resp. feature selection), up to 18.2% (resp. 32.6%). (c) Batch enrichment is 5.94X faster than the baselines on IMDB on average. (4) The incremental method beats the batch one when updates to D and G are up to 20%, and is 6.28X faster when updates $|\Delta G| = 5\%|G|$.

We discuss related work in Section 7 and future work in Section 8.

2 PRELIMINARIES

In this section, we review basic notations, ER and HER.

Relations. Consider a relation schema $R = (\mathsf{id}, A_1, \ldots, A_n)$, where A_i is an attribute ($i \in [1, n]$), and id is an entity id as introduced by Codd [39], such that each tuple of R represents an entity of type τ with identity id. A relation D of R is a set of tuples of schema R.

Knowledge graphs. Following [68], we represent a knowledge graph as $G = (V, E, L)$. Here (a) V is a finite set of vertices representing entities, (b) $E \subseteq V \times V$ consists of edges representing relationships between entities; and (c) for each vertex $v \in V$, $L(v)$ is its feature or value, and for each edge $e \in E$, $L(e)$ is its label. Between a pair (v, v') in V, there are possibly multiple edges carrying distinct labels.

A *path* ρ *from a vertex* v_0 in graph G is $\rho = (v_0, v_1, \ldots, v_l)$ such that (v_{i-1}, v_i) is an edge in E for $i \in [1, l]$. The *length of* ρ is the number l of edges on ρ. A path is *simple* if each vertex appears on ρ at most once. We consider simple paths, simply referred to as paths.

Entity resolution. Given a relation D, entity resolution (ER) is to identify all pairs of tuples in D that refer to the same real-life entity. It returns a set of pairs (t_1, t_2) of tuples of D that are identified as *matches*. If t_1 does not match t_2, (t_1, t_2) is referred to as a *mismatch*.

A number of ER methods have been developed, based on ML [23, 45, 74, 86, 95, 102, 134], logic rules [24, 27, 32, 48, 61, 79, 127] and hybrid of the two [28, 41, 56]. We focus on ML-based ER models, which take quadratic-time (after the relevant ML models are trained).

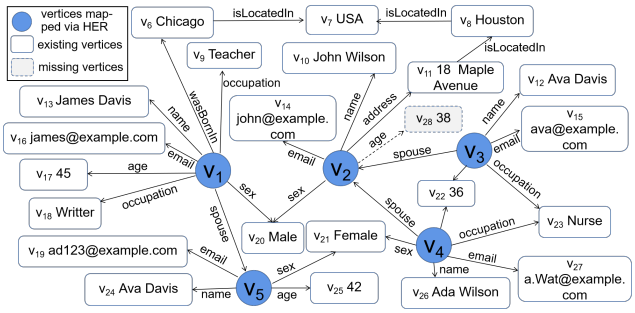


Figure 1: An example knowledge graph

*Heterogeneous entity resolution (*HER*).* HER is to identify entities across a relation and a graph. It is defined as a *mapping* f_{HER} that given a graph G and a set D of tuples of schema R, computes a set:

$$f_{\mathsf{HER}}(D, G) = \{(t, v) \mid t \in D, v \in V \text{ in } G, t \Rightarrow v\}.$$

Here $t \Rightarrow v$ denotes that tuple t and vertex v make a *match*, *i.e.,* t and v refer to the same entity. For each $t \in D$, there can be multiple vertices v in G that match t. We refer to f_{HER} as the HER *mapping*.

Several methods are in place for HER, *e.g.,* rule-based JedAI [98], parametric simulation [51], and ML-based Silk [72], MAGNN [58] and EMBLOOKUP [19]. In particular, it is in $O(|D||G|)$ time to compute all HER matches across D and G by parametric simulation [51].

We assume that schema R has enough information for a well-designed HER mapping f_{HER} to map tuples of R to entities in G.

Example 2: Consider a KG G in Figure 1 for tuples in Table 1 (*e.g.,* derived from social networks [65]), where t_i matches v_i ($i \in [1, 5]$).

HER differs from ER. (a) It identifies tuples and vertices across a relation and a graph, while ER matches tuples in a relation; (b) an attribute in t may map to a path in G, *e.g.,* city of t_2 vs. $\rho = (v_2, v_{11}, v_8)$ in G; (c) not every attribute in t can find a matching path in G, and vice versa, *e.g.,* v_4 has occupation "Nurse", which finds no corresponding attribute in Table 1; and (d) t and v often have different descriptions for the same property, *e.g.,* gender of t_2 vs. sex of v_2. □

3 A SCHEME FOR RELATION ENRICHMENT

In this section, we first formulate the enrichment problem and incremental enrichment problem for ER (Section 3.1). We then settle the complexity of the enrichment problems (Section 3.2). After this, we propose enrichment scheme ENRICH (Section 3.3).

3.1 Relation Enrichment Problem

Given a relation schema $R = (\bar{A})$, where $\bar{A}$ is a set $(\mathsf{id}, A_1, \ldots, A_n)$ of attributes, consider a relation D of R, a KG G, and an ER model $\mathcal{A}_{\mathsf{ER}}$.

Accuracy. We first present how to measure the accuracy improvement on ER models, in terms of Precision, Recall and F_1.

Following Codd [39], consider tuples for representing a (countably infinite) set $\mathcal{E}$ of real-world entities. For tuples t in instance D of schema $R = (\bar{A})$, there exists a mapping f from each tuple ID in D to $\mathcal{E}$ such that $f(t.\mathsf{id}) = e$, *i.e.,* for each tuple t in D, $f(t.\mathsf{id})$ is the entity represented by t (such mapping is usually implicitly assumed in ER). Then the accuracy of $\mathcal{A}_{\mathsf{ER}}$ on D is traditionally measured in terms of $\mathsf{F}_1 = \frac{2 \cdot \mathsf{Precision} \cdot \mathsf{Recall}}{\mathsf{Precision} + \mathsf{Recall}}$. Here Precision is the ratio of *pairs of distinct tuples* that are correctly identified to all identified tuple pairs, *i.e.,* $\mathsf{Precision} = \frac{|\{(t,s) \mid t,s \in D,\ \mathcal{A}_{\mathsf{ER}}(t,s)=\mathsf{true},\ f(t.\mathsf{id})=f(s.\mathsf{id})\}|}{|\{(t,s) \mid t,s \in D,\ \mathcal{A}_{\mathsf{ER}}(t,s)=\mathsf{true}\}|}$ for distinct t and s, and Recall is the ratio of correctly identified tuple pairs to all tuple pairs that refer to the same real-world entity,

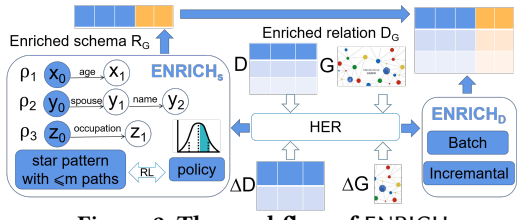


Figure 2: The workflow of ENRICH

i.e., $\mathsf{Recall} = \frac{|\{(t,s) \mid t,s \in D,\ \mathcal{A}_{\mathsf{ER}}(t,s)=\mathsf{true},\ f(t.\mathsf{id})=f(s.\mathsf{id})\}|}{|\{(t,s) \mid t,s \in D,\ f(t.\mathsf{id})=f(s.\mathsf{id})\}|}$.

Example 3: Consider Table 1 of schema $R = (\bar{A}) = (\mathsf{name}, \mathsf{gender}, \mathsf{email}, \mathsf{address}, \mathsf{city})$, where D has 5 tuples, $t_1[\bar{A}]$-$t_5[\bar{A}]$, and the mapping f is shown in the table. Here assume that $\mathcal{A}_{\mathsf{ER}}$ makes an FP prediction and an FN prediction as stated in Example 1. The precision of $\mathcal{A}_{\mathsf{ER}}$ on D is $\mathsf{Precision} = \frac{0}{1}$ since the only distinct pair predicted true by $\mathcal{A}_{\mathsf{ER}}$ is (t_3, t_5) (which is a FP). Similarly, $\mathsf{Recall} = \frac{0}{1}$ since the only true match (t_3, t_4) is not identified (due to the FN). □

As shown above, $\mathcal{A}_{\mathsf{ER}}$ on D is not accurate, for the lack of attributes. To improve it, we aim to enrich schema $R = (\bar{A})$ to $R_G = (\bar{A}, \bar{B})$, where $\bar{A}$ copies the attributes of R, $\bar{B}$ is a set of at most m attributes extracted from graph G, and m is the "budget" for extending schema R. Intuitively, we want to extend D and create an instance D_G of schema R_G, such that for each t in D, we have exactly one *enriched tuple* $t_G \in D_G$, where $t_G.\mathsf{id} = t.\mathsf{id}$, $t_G[\bar{A}] = t[\bar{A}]$ and $t_G[\bar{B}]$ is the partial tuple extracted from G. We refer to R_G as the *enriched schema* of R with G, and to D_G as the *enriched relation* of D with G.

Example 4: Assume that $R_G = (\bar{A}, \bar{B})$ where $\bar{B} = (\mathsf{spouse_name})$. After enriching R to R_G with an additional attribute spouse_name, the FN (*i.e.,* (t_3, t_4)) is reduced, as stated in Example 1, improving Precision of $\mathcal{A}_{\mathsf{ER}}$ on D_G to $\frac{1}{2}$, since $\mathcal{A}_{\mathsf{ER}}$ predicts true for both (t_3, t_5) (which is the FP) and (t_3, t_4), where only the latter one is correctly identified. Similarly, Recall is also improved to $\frac{1}{1}$ since the only true match (t_3, t_4) in Table 1 is correctly identified. □

We use the difference between the F_1 of $\mathcal{A}_{\mathsf{ER}}$ on D_G and on D, denoted by $\Delta\mathsf{F}_1$, as *improvement of* $\mathcal{A}_{\mathsf{ER}}$ *on* D *via* D_G. The difference of Precision/Recall can also be used, depending on application needs.

Problems. We now state the *enrichment problem* for ER model $\mathcal{A}_{\mathsf{ER}}$.

- *Input*: $R = (\bar{A})$, D and G as above, and a positive integer m.
- *Output*: (a) An enriched schema $R_G = (\bar{A}, \bar{B})$ of R with G such that R is extended with at most m attributes $\bar{B}$ extracted from G; and (b) an enriched relation D_G of D with G.
- *Objective*: To maximize the improvement of $\mathcal{A}_{\mathsf{ER}}$ on D via D_G.

Here ER is conducted by the same $\mathcal{A}_{\mathsf{ER}}$ on both D and D_G.

The enrichment problem can be sub-divided into two problems: (1) *schema enrichment*, to deduce enriched schema R_G, and (2) *data enrichment*, to compute enriched relation D_G after R_G is in place.

Incremental enrichment problem. Real-life data is constantly changed by small updates. Consider updates to D and G. Updates to relation D consist of deleted/inserted tuples, denoted by ΔD; note that modifications to a tuple t can be regarded as deleting t followed by inserting a tuple with the changed values. Graph updates, denoted by ΔG, consist of edge insertions/deletions. Note that vertex updates are a dual [80] and can be handled similarly; and the change to an edge label can be seen as the deletion of an existing edge, followed by the insertion of a new one with the updated label. We use $G \oplus \Delta G$ to denote graph G updated by ΔG; similarly for $D \oplus \Delta D$.



Through the Fairness Lens: Experimental Analysis and Evaluation of Entity Matching

Nima Shahbazi
University of Illinois Chicago
nshahb3@uic.edu

Nikola Danevski
University of Rochester
ndanevsk@u.rochester.edu

Fatemeh Nargesian
University of Rochester
fnargesian@rochester.edu

Abolfazl Asudeh
University of Illinois Chicago
asudeh@uic.edu

Divesh Srivastava
AT&T Chief Data Office
divesh@research.att.com

ABSTRACT

Entity matching (EM) is a challenging problem studied by different communities for over half a century. Algorithmic fairness has also become a timely topic to address machine bias and its societal impacts. Despite extensive research on these two topics, little attention has been paid to the fairness of entity matching.

Towards addressing this gap, we perform an extensive experimental evaluation of a variety of EM techniques in this paper. We generated two social datasets from publicly available datasets for the purpose of auditing EM through the lens of fairness. Our findings underscore potential unfairness under two common conditions in real-world societies: (i) when some demographic groups are over-represented, and (ii) when names are more similar in some groups compared to others. Among our many findings, it is noteworthy to mention that while various fairness definitions are valuable for different settings, due to EM's class imbalance nature, measures such as positive predictive value parity and true positive rate parity are, in general, more capable of revealing EM unfairness.

PVLDB Reference Format:
Nima Shahbazi, Nikola Danevski, Fatemeh Nargesian, Abolfazl Asudeh, and Divesh Srivastava. Through the Fairness Lens: Experimental Analysis and Evaluation of Entity Matching. PVLDB, 16(11): 3279 - 3292, 2023.
doi:10.14778/3611479.3611525

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/UIC-InDeXLab/fair_entity_matching.

1 INTRODUCTION

Entity matching (EM) seeks to match pairs of entity records from (the same or different) data sources that refer to the same real-world entity. EM is very useful in many applications domains, including (a) healthcare, where matching of patient records from different healthcare facilities (e.g., emergency rooms, hospitals, etc.) can be used to determine if they refer to the same real-world person; (b) airline security, where airline passenger records are matched against no-fly list records to identify people who should be prevented from boarding flights or should undergo additional screening; (c) e-commerce, where product records from different retailers' websites can be matched to identify popular products and fraudulent knockoffs; and so on.

EM is a challenging problem that has been extensively investigated for over half a century by different communities, e.g., statistics, databases (DB), natural language processing (NLP), and machine learning (ML), resulting in a variety of techniques proposed in the literature for addressing this problem. These challenges arise because entities in autonomous data sources can be represented in a variety of ways (e.g., highly structured records versus textual descriptions), using different conventions (e.g., the many ways in which person names and postal addresses are represented), data quality issues (e.g., misspellings, missing values), and so on. A consequence is that, despite significant advances in recent years (especially with recent neural techniques like Ditto [36]), EM techniques still result in both false positives (non-matching record pairs that are declared as matches) and false negatives (matching record pairs that are declared as non-matches). These errors can have serious consequences in practice, as seen in the following examples.

Example 1: (No-fly list) Consider the airline security application, which aims to identify passengers that are likely to be dangerous (e.g. terrorists) for screening and potentially preventing them from boarding the flights. Using a dataset of criminal records called the no-fly list, passenger names (and other information) are matched against the no-fly list for this purpose. False positives in airline security can lead to significantly inconveniencing passengers. On the other hand, false negatives can result in known terrorists being permitted to board flights with undesirable consequences. Due to historical biases, the no-fly list datasets could over-represent some minority groups in comparison to society's population distribution. This, as we shall evaluate in our experiments, can result in higher false positive rates for those demographic groups. Another potential issue is that some demographic groups have more similar names. Hence, passengers from those groups may have a higher chance of having the same or similar information to those of known terrorists, which in turn will cause higher false positive rates for them. □

In Example 1, getting (falsely) matched is harmful. We next show an example where not getting matched can be harmful.

Example 2: (High-value customers list) Upselling to potentially high-value customers is critical for many businesses such as the fashion industry, airlines, and tourism. Suppose business A acquires

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 11 ISSN 2150-8097.
doi:10.14778/3611479.3611525

a list L of names of high-value customers from other businesses. Business A uses EM techniques to match its own customers against list L and sends exclusive offers to upsell to the matching cases in order to prioritize them. Unlike the previous example, in this case, over-representation in the high-value customer's list L is beneficial. However, unlike the no-fly list, the privileged group(s) are likely to be over-represented and minority groups under-represented in the list L. As a result, a lower (true and false) positive match rate is expected for minority groups and they end up receiving fewer exclusive offers, resulting in biased advertising [4, 9, 40]. □

When such disparities (e.g., false positives) occur in a systematic way for some demographic (sub-)groups, thereby disadvantaging them over others, concerns about the *fairness* of EM techniques arise. While the fairness of ML models has been the topic of much recent work in the literature [12, 23, 25, 25, 28, 34, 61, 62, 64], not much attention has been paid to the fairness of EM techniques.

In this paper, we seek to address this gap in the literature and perform an extensive experimental evaluation and analysis of a variety of EM techniques on a range of datasets through the *fairness lens*. Traditionally, blocking may precede matching to reduce the space of possible matching candidates from quadratic to sub-quadratic, e.g., linear. A rich body of research focuses on blocking algorithms [19, 35, 46, 47]. In this paper, our goal is to audit off-the-shelf entity matching systems used in practice. As such, our evaluation and analysis are performed on end-to-end matching systems which may include their own built-in blocking algorithms.

Summary of Contributions: In summary, we make the following technical contributions in this paper:

- Given the pairwise nature of EM, we propose the use of *single fairness* and *pairwise fairness* to evaluate entity matchers. We adopt 11 popular fairness measures from the literature for this task and analyze their suitability for EM.
- We select a suite of 13 EM techniques (including 1 declarative rule-based technique, 7 non-neural ML techniques, and 5 neural ML techniques) and 6 benchmark datasets (including 2 structured datasets, 2 textual datasets, and 2 dirty datasets) that have been used in prior work on entity matching for fairness evaluation. Using publicly available individual-level data, we also created and used two semi-synthetic matching *social datasets* for fairness evaluation. These datasets are shared publicly as benchmarks for auditing the fairness of matchers.
- We evaluated all combinations of EM techniques, datasets, and fairness measures and analyzed the outcomes. We classified the results into four cases based on whether an (EM technique, dataset, or fairness measure) yielded (i) accurate or inaccurate matching results, and (ii) fair or unfair matching results.
  Some of our findings in this study are as follows:
  - Our results on social data confirm matching unfairness when (i) there are higher similarities among records of a certain group, (ii) the representation of demographic groups in data is biased.
  - Our results underscore that responsible EM requires training data that is representative of different possibilities from various (demographic) groups.
  - While different fairness measures are valuable for different settings, due to the class-imbalance property of EM, measures such as *positive predictive value parity* and *true positive rate parity* are more capable of revealing EM unfairness.
  - Significantly relying on proxy attributes such as name, can cause unfairness in non-neural models. On the other hand, relying on pre-trained language models and embeddings, or not fully considering the dataset structure can cause unfairness in neural matchers.

  We use our findings and lessons learned to put together a set of rules of thumb for responsible entity matching (Table 8).

2 RELATED WORK

Fairness in entity resolution (ER) has briefly been studied in the literature. In [24], a constraint-based formulation for fairness is proposed to mitigate bias in ER tasks by ensuring that all (sub-)groups have the same opportunity to be resolved. Furthermore, [38] proposes a (sub-)group-based training for different ethnicities in order to increase both accuracy and fairness in SVM-based ER which is consistent with our suggestion to use ensemble learning for EM. Finally, in a parallel work [43], the authors propose an AUC-based fairness definition for EM and ER tasks and try to resolve the bias issues through a data augmentation solution. To the best of our knowledge, we are the first to comprehensively audit off-the-shelf entity matching models for fairness and propose proper measures, datasets, and comparison angles fitting the problem settings given the inherent differences with typical machine learning tasks.

3 FAIRNESS EVALUATION FRAMEWORK

3.1 Background

Given two sets of records A and B, the EM problem is to identify all correspondences between record pairs in $A \times B$ that correspond to the same real-world entity. A correspondence $c = (e_i, e_j, s)$ interrelates two records e_i and e_j with a confidence value $s \in [0, 1]$ that indicates the similarity of e_i and e_j or the confidence of a matcher about e_i and e_j referring to the same entity [33]. To decide whether the record pair of $c = (e_i, e_j, s)$ is a *match* or *non-match*, matchers often apply a threshold on s [11, 60]. We decouple the choice of a threshold from the outcome of the matching and consider the outcome of an EM task as pairs of matching and non-matching records. Formally, we consider the following EM problem:

Definition 1 (Entity Matching Problem). *Consider two sets of records A and B. For every pair of records $(e_i, e_j) \in A \times B$, let y_{ij} be the ground-truth label indicating if e_i and e_j refer to the same entity. Given all pairs $(e_i, e_j) \in A \times B$, the EM problem is to predict y_{ij} with a label h_{ij}. That is, h_{ij} refers to the decision of the matcher about the label of e_i and e_j (match or non-match).*

In a fairness-sensitive setting, records are accompanied by sensitive attributes (e.g. `gender`, `country`, `race`, etc.). Let $\mathcal{A} = \{A_1, \ldots, A_n\}$ be the sensitive attributes, $dom(A_i)$ be the domain of A_i, and $\mathcal{G} = \{g_1, \ldots, g_m\}$ be the set of all groups of interest, i.e. $\mathcal{G} = \bigcup_{A_i \in \mathcal{A}} dom(A_i)$. The mapping $L(e_i)$ relates a record to its associated groups $G_i \subseteq \mathcal{G}$. In other words, G_i is the group that e_i belongs to. Given two sets of records A and B and the set $[(e_i, e_j, G_i, G_j, h_{ij}, y_{ij})]_{\forall (e_i, e_j) \in A \times B}$, we would like to audit the fairness of a matcher with respect to groups.

3.2 Single and Pairwise Lens

3.2.1 Group Selection. The first step in auditing an entity matcher for fairness is identifying meaningful (sub-)groups in sensitive attributes. An input dataset to a matcher $\mathcal{M}$ includes record ids, the value ((sub-)group) of each record for sensitive attributes, the decisions of $\mathcal{M}$, as well as true labels for the record pairs. Depending on the type, cardinality, and the number of sensitive attributes, multiple fairness cases may happen that are presented in Table 1.

The space of groups for a single attribute with binary or multiple values is the domain of the corresponding attribute. In multiple-attribute settings, we can define intersectional subgroups, as the cartesian product of group values. More details about intersectional subgroups are provided in the technical report [50].

3.2.2 Single and Pairwise Fairness Evaluation. Given the pairwise nature of EM tasks, there are two ways to audit entity matchers:

- *Single Fairness:* The performance of a matcher is evaluated for one group s against either record in a pair. Given a correspondence $c = (e_i, e_j, h, y)$ and a group s of interest, c is legitimate, if either e_i or e_j belong to group s.
- *Pairwise Fairness:* The performance of a matcher is evaluated for a pair of groups s, s' against both records in a pair. Given a correspondence $c = (e_i, e_j, h, y)$ and a pair of groups (s, s') of interest, c is legitimate, if e_i belongs to s and e_j belongs to s', or vice versa. From an encoding perspective, we concatenate the encodings of groups s and s' into a vector c and the encodings (explained in the technical report [50]) of e_i and e_j into a vector e and validates vector e belongs to c with both directions of $\langle s, s' \rangle$ and $\langle s', s \rangle$.

We consider the EM task to be symmetric in single and pairwise fairness definitions. We remark that these definitions can be extended to ordered single and ordered pairwise fairness where the groups are defined on left or right records. In this paper, we focus on non-directional single and pairwise fairness.

3.3 Correctness

The correctness of a matcher measures how well its matching predictions conform with the ground-truth. Given a test dataset with correspondences of $t = (e_i, e_j, h, y)$, where h is a binary variable indicating the result of EM (*match* or *non-match*) for records with encodings e_i and e_j, and y is a binary variable indicating the ground-truth for matching, we profile predictions of h using the numbers of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN), respectively. Unlike a classification task, in the confusion matrix of a matching task, the result is counted both for the group(s) of e_i and the group(s) of e_j. For further explanations, please refer to the technical report [50].

3.4 Fairness Measures

Similar to many of the existing works on algorithmic fairness and responsible data science [5, 15, 16, 25, 28, 42, 52, 54], we use the terms bias and unfairness interchangeably to refer to when a matcher is not fair. At a high level, fairness definitions can be viewed from three perspectives: group, subgroup, and individual fairness [12]. The most granular notion of fairness is individual fairness that requires similar outcomes for similar individuals [23]. The more popular perspective of fairness, (sub)group fairness, requires similar treatment for different (sub)groups. A model/algorithm satisfies some fairness constraints if it has equal or similar performance (according to some fairness measure) on different (sub)groups. *The focus of this paper is on (sub)group fairness.* Most of the group fairness measures belong to one of the following four categories [6, 12]. (1) *Independence* requires independence of analysis outcome from demographic groups. (2) *Separation* requires independence of the outcome from demographic groups conditioned on the target variable. (3) *Sufficiency* requires independence of the target variable from demographic groups conditioned on the outcome. (4) *Causation* requires that in a counterfactual world, the decision would not change had the individual belonged to a different demographic group. We do not consider Causal fairness in our audit. In Table 2, we present our suite of fairness measures, adapted from the notions of fairness in classification [12], for auditing an entity matcher $\mathcal{M}$ for a set $\mathcal{G}$ of (sub)groups.

3.5 Selecting Fairness Measures for EM

Depending on the context of an EM task at hand, proper fairness measures should be employed. Besides, a major difference between EM and regular classification tasks is that the input to EM tasks is a pair of records. Due to its pairwise matching nature, *class imbalance* is a distinguishing property of EM, compared to regular classification tasks. To better explain this, let us consider a toy example, where two data sources D and D' contain exactly the same set of n records. Each pair of records $e \in D$ and $e' \in D'$ is passed as an input to an entity matcher. In this setting, only n of the n^2 pairs are matches, and the others are non-matches. In other words, the probability a random pair is a match is as low as $\frac{1}{n}$. Class imbalance is also a challenge in some of classification problems but the degree of imbalance is often constant while in EM tasks, it is as high as $O(n)$. Indeed, blocking techniques [46] can help in reducing the extreme class imbalance. Even after blocking, a high class imbalance is expected for EM tasks.

Guide for Practitioners: Let us consider the fairness measures in Table 2. Which measures to choose depends on the downstream task and the problem context. Therefore, practitioners should choose measures depending on the importance of TPs (true match), FPs, FNs, and TNs in the problem context. For example, among the fairness measures, *statistical parity* does not consider the ground-truth labels and requires equal match ratios from different groups, independent of whether they really are a match or not. As a result, this measure does not seem reasonable for deduplication tasks using EM. On the other hand, it may be useful to ensure equal representation of different groups when using EM for joining tables.

When the input to the EM task is imbalanced and most of the pairs are non-matches, some measures are more capable of revealing the unfairness of matchers. First, note that even a matcher that marks all pairs as non-matches has high accuracy in this setting. Subsequently, accuracy parity may not reveal the unfairness. Similarly, measures such as FPRP and TNRP may fail to reveal unfairness in detecting true matches. In these settings, the fairness measure for successfully discovering these events is **Positive Predictive Value Parity (PPVP)**. Another important measure in this context is **True Positive Rate Parity (TPRP)**, a.k.a **Equal Opportunity**, which focuses on correct match predictions among the (rare) true